Routing in Vite with React Router

This guide explains how to implement routing in a Vite-powered React application and explores the relationship between the build tool and the routing library.

Why Vite Uses React Router

The answer lies in the **separation of concerns**:

1. **Vite is a Build Tool:** Its primary job is to serve your code during development with extreme speed and bundle it for production. It is "unopinionated" about navigation.
2. **React is a Library:** React focuses on the UI and component tree. It does not include a built-in router to keep the core library lightweight.
3. **React Router is a specialized library:** It handles the complex logic of synchronizing the browser URL with the rendered UI.

Recommended File Structure

When using routing, a clean directory structure is essential for scalability. Here is the standard "Feature-based" or "Type-based" structure for a Vite React project:

```
my-vite-app/
├── public/              # Static assets (favicons, etc.)
├── src/
│   ├── assets/          # Images, global CSS
│   ├── components/      # Reusable UI components (Navbar, Button)
│   │   └── Navbar.jsx
│   ├── layouts/         # Shared wrappers (MainLayout, AuthLayout)
│   │   └── RootLayout.jsx
│   ├── pages/           # Route-specific view components
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   └── NotFound.jsx
│   ├── App.jsx          # Main application logic/traditional routes
│   ├── main.jsx         # Entry point (Router setup here)
│   └── index.css        # Global styles
├── index.html           # The mounting point for the React app
├── package.json
└── vite.config.js
```

Installation

To get started, add the web-specific version of React Router to your project:

```
npm install react-router-dom
```

Implementation Patterns

1. The Modern Approach: `createBrowserRouter`

This is the recommended approach for modern React applications. It is usually implemented directly in `main.jsx`.

Main Entry Point (`src/main.jsx`):

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import RootLayout from './layouts/RootLayout';

const router = createBrowserRouter([
  {
    path: "/",
    element: <RootLayout />, // Shared UI like Header/Footer
    children: [
      {
        path: "/",
        element: <Home />,
      },
      {
        path: "/about",
        element: <About />,
      },
    ],
  },
]);

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

2. The Traditional Approach: `<BrowserRouter>`

If you prefer a declarative structure inside your components.

App Structure (`src/App.jsx`):

```
import { Routes, Route, Link } from "react-router-dom";
import Home from './pages/Home';
import About from './pages/About';

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<h1>404 Not Found</h1>} />
      </Routes>
    </div>
  );
}
```

Navigation Basics

Always use the `<Link>` component instead of anchor `<a>` tags to prevent full page reloads.

```
import { Link, useNavigate } from "react-router-dom";

function Navigation() {
  const navigate = useNavigate();

  return (
    <>
      <Link to="/profile">Go to Profile</Link>
      <button onClick={() => navigate("/logout")}>Logout</button>
    </>
  );
}
```

Key Benefits of this Setup

- **Lazy Loading:** Easily split routes using `React.lazy()` to optimize performance.
- **HMR (Hot Module Replacement):** Vite updates page components without losing the current routing state.
- **Single Page Application (SPA) feel:** Navigating is instantaneous as only the components change, not the whole page.